Indian Institute of Technology Madras

Department of Data Science and Artificial Intelligence

Vehicle Insurance Cross-Sell Prediction

An End-to-End MLOps System

Course: DA5402 — Machine Learning Operations

Project Type: Final Project

Author:	Ganesh Mula
Student ID:	da25m019
Email:	ganeshmula02@gmail.com
GitHub:	Ganesh2024/DA5402-ML_Ops_Final_Project

Date: April 2026

Abstract

This report presents an end-to-end MLOps system for vehicle insurance cross-sell prediction. The system covers the complete machine learning lifecycle: data ingestion from MongoDB Atlas, schema validation, feature engineering with SMOTEENN balancing, multi-model training (RandomForest, ExtraTrees, GradientBoosting) with MLflow experiment tracking, automated model evaluation against production, S3-based model storage, and deployment via FastAPI on AWS EC2 using Docker containers. Pipeline orchestration is implemented with Apache Airflow, and CI/CD is managed through GitHub Actions. The best model (GradientBoosting) achieves an F1 score of 0.949.

Contents

1 Introduction **2**
- 1.1 Problem Statement . . . 2
- 1.2 Dataset . . . 2
- 1.3 MLOps Objectives . . . 2

2 System Architecture **2**
- 2.1 Technology Stack . . . 3

3 Training Pipeline **3**
- 3.1 Stage 1 — Data Ingestion . . . 3
- 3.2 Stage 2 — Data Validation . . . 3
- 3.3 Stage 3 — Data Transformation . . . 3
- 3.4 Stage 4 — Model Trainer . . . 4
- 3.5 Stage 5 — Model Evaluation . . . 4
- 3.6 Stage 6 — Model Pusher . . . 4

4 Airflow Pipeline Orchestration **4**
- 4.1 Successful Pipeline Run . . . 4
- 4.2 Pipeline Failure Propagation . . . 5

5 MLflow Experiment Tracking **6**
- 5.1 Tracked Information . . . 7
- 5.2 Run Details . . . 7
- 5.3 Model Metrics Comparison . . . 7
- 5.4 Model Parameters Comparison . . . 8
- 5.5 Parallel Coordinates Plot . . . 8
- 5.6 Model Performance Results . . . 8

6 CI/CD Pipeline **9**
- 6.1 Pipeline Overview . . . 9
- 6.2 Continuous Integration . . . 10
- 6.3 Continuous Deployment . . . 10

7 Web Application **10**
- 7.1 FastAPI Endpoints . . . 11
- 7.2 Prediction Flow . . . 11

8 Software Engineering **11**
- 8.1 Design Principles . . . 11
- 8.2 Logging . . . 11
- 8.3 Exception Handling Pattern . . . 11

9 Known Limitations & Future Work **12**

10 Conclusion **12**

AI Usage Disclosure **13**

1. Introduction

1.1. Problem Statement

The goal of this project is to predict whether an existing health insurance customer would be interested in purchasing vehicle insurance. This is a binary classification problem where the target variable `Response` takes values 1 (Interested) or 0 (Not Interested).

1.2. Dataset

The dataset contains 381,109 customer records with 12 features including demographic information, vehicle details, and policy information. The data is stored in MongoDB Atlas and accessed via a Python-based data pipeline.

Table 1: Dataset Features

Feature	**Type**	**Description**
Gender	Categorical	Male / Female
Age	Numerical	Customer age in years
Driving_License	Binary	0 = No, 1 = Yes
Region_Code	Numerical	Customer's region (0–52)
Previously_Insured	Binary	Already has vehicle insurance
Vehicle_Age	Categorical	<1 Year / 1-2 Year / >2 Years
Vehicle_Damage	Categorical	Yes / No
Annual_Premium	Numerical	Annual premium amount
Policy_Sales_Channel	Numerical	Channel code (1–163)
Vintage	Numerical	Days associated with company
Response	**Target**	**0 = No, 1 = Yes**

1.3. MLOps Objectives

- Automate the complete ML lifecycle from raw data to deployed model
- Track all experiments with MLflow for reproducibility and comparison
- Orchestrate pipeline stages with Apache Airflow with proper failure handling
- Deploy via CI/CD using GitHub Actions, Docker, AWS ECR, and EC2
- Expose predictions via a RESTful FastAPI web application

2. System Architecture

The system is organized into five logical layers:

1. **Data Layer:** MongoDB Atlas storing 381,109 insurance customer records
2. **Training Layer:** 6-stage sequential pipeline orchestrated by Airflow
3. **Tracking Layer:** MLflow with SQLite backend for experiment logging

4. **Storage Layer:** AWS S3 for trained model artifact storage
5. **Deployment Layer:** Docker + AWS ECR + EC2 + GitHub Actions CI/CD

2.1. Technology Stack

Table 2: Technology Stack

Layer	Technology	Rationale
Data Store	MongoDB Atlas	Cloud-native, flexible schema
ML Framework	scikit-learn	Industry standard ensemble models
Imbalance Handling	SMOTEENN	Combined over+undersampling
Serialization	dill	Handles complex sklearn Pipelines
Experiment Tracking	MLflow 3.x	Open-source, rich UI
Orchestration	Apache Airflow 3.x	DAG-based pipeline management
Web Framework	FastAPI	Async, high performance
Container	Docker	Environment reproducibility
Image Registry	AWS ECR	Tight EC2 integration
Compute	AWS EC2 T2 Medium	Scalable cloud compute
Model Storage	AWS S3	Durable, cheap object storage
CI/CD	GitHub Actions	Native git integration

3. Training Pipeline

The training pipeline consists of six sequential stages. Each stage produces an *Artifact dataclass* passed to the next stage.

3.1. Stage 1 — Data Ingestion

Connects to MongoDB Atlas using `MongoDBClient` (singleton pattern), fetches all 381,109 records, and performs a 75/25 train/test split. Outputs `train.csv` and `test.csv` to a timestamped artifact directory.

3.2. Stage 2 — Data Validation

Validates ingested data against `config/schema.yaml`: column count (12 required), numerical column presence, and categorical column presence. Returns `validation_status = True/False`. If False, all downstream Airflow tasks are marked `upstream_failed`.

3.3. Stage 3 — Data Transformation

Applies transformations in sequence:

1. Gender encoding: Female → 0, Male → 1
2. ID drop: Removes `id` and `_id` columns (MongoDB artifacts)
3. Dummy encoding: `pd.get_dummies(drop_first=True)`

4. Column rename: `Vehicle_Age_< 1 Year` → `Vehicle_Age_lt_1_Year`
5. Scaling: StandardScaler on [Age, Vintage], MinMaxScaler on [Annual_Premium]
6. SMOTEENN: Handles class imbalance (minority class ≈12%)

3.4. Stage 4 — Model Trainer

Trains three candidate models and logs each to MLflow:

Table 3: Model Hyperparameters

Parameter	RandomForest	ExtraTrees	GradientBoosting
n_estimators	100	100	100
max_depth	10	10	10
min_samples_split	7	7	7
min_samples_leaf	6	6	6
criterion	entropy	entropy	—
random_state	101	101	101

Best model selected by F1 score and saved as `model.pkl` wrapping both the preprocessing pipeline and the trained classifier.

3.5. Stage 5 — Model Evaluation

Compares the new model against the current production model stored in S3. Accepts if improvement ≥ 0.02. Logs evaluation metrics to MLflow.

3.6. Stage 6 — Model Pusher

Uploads the accepted model to AWS S3 as `model.pkl`.

4. Airflow Pipeline Orchestration

Apache Airflow 3.x orchestrates the training pipeline as a DAG using `PythonOperator` for each stage with `trigger_rule='all_success'`.

4.1. Successful Pipeline Run

Figure 1 shows all six tasks completing successfully.

Figure 1: Airflow DAG — All 6 tasks succeeded (green)

4.2. Pipeline Failure Propagation

Figure 2 demonstrates failure propagation. When `model_training` fails, all downstream tasks are automatically marked `upstream_failed` without executing.

Figure 2: Airflow DAG — model_training failed, downstream tasks blocked (orange)

5. MLflow Experiment Tracking

MLflow tracks all experiments under `VehicleInsurance` using a SQLite backend.

5.1. Tracked Information

Table 4: MLflow Tracked Information per Run

Run Name	Parameters	Metrics
RandomForest	model_type, n_estimators, max_depth, criterion, min_samples_split, min_samples_leaf, random_state	accuracy, f1_score, precision, recall
ExtraTrees	Same as RandomForest	accuracy, f1_score, precision, recall
GradientBoosting	model_type, n_estimators, max_depth, min_samples_split, min_samples_leaf, random_state	accuracy, f1_score, precision, recall
BestModel_*	best_model, selection_metric	best_f1, best_accuracy
Model_Evaluation	model_accepted, threshold	new_f1, prod_f1, improvement

5.2. Run Details

Run details

Run ID:	db088ce024fa4418b186bbd790878cad	c91d76d88112457d958011819b54b706	342b27fc13c442b686a776ea16e08acd
Run Name:	GradientBoosting	ExtraTrees	RandomForest
Start Time:	04/29/2026, 08:50:33 AM	04/29/2026, 08:44:02 AM	04/29/2026, 08:43:36 AM
End Time:	04/29/2026, 08:50:37 AM	04/29/2026, 08:44:06 AM	04/29/2026, 08:43:41 AM
Duration:	4.3s	3.7s	5.0s

Figure 3: MLflow — Individual run details with metrics and parameters

5.3. Model Metrics Comparison

Metrics

Show diff only

accuracy	0.945	0.913	0.924
f1_score	0.949	0.923	0.932
precision	0.925	0.866	0.882
recall	0.975	0.989	0.988

Figure 4: MLflow — Metrics comparison: GradientBoosting vs ExtraTrees vs RandomForest

5.4. Model Parameters Comparison

Parameters

Show diff only

criterion		entropy	entropy
max_depth	10	10	10
min_samples_leaf	6	6	6
min_samples_split	7	7	7
model_type	GradientBoosting	ExtraTrees	RandomForest
n_estimators	100	100	100
random_state	101	101	101

Figure 5: MLflow — Parameter comparison across three models

5.5. Parallel Coordinates Plot

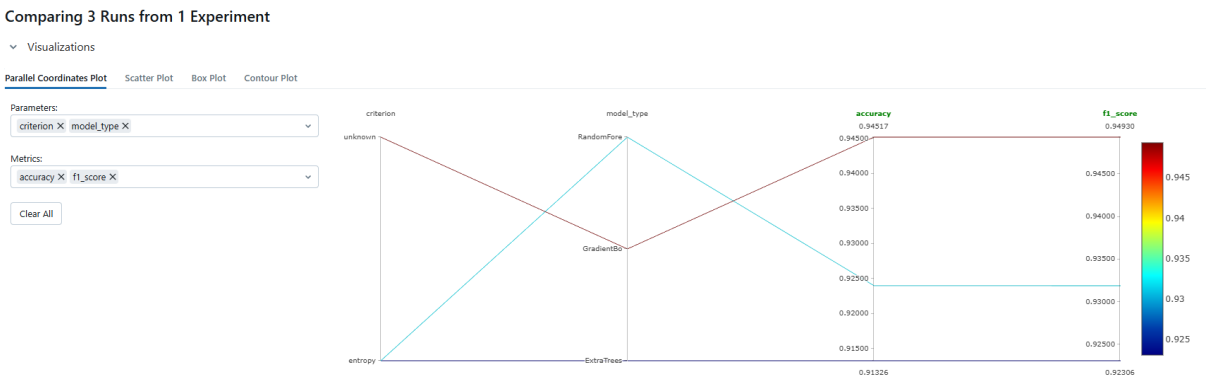

Figure 6: MLflow — Parallel coordinates: model_type vs accuracy vs f1_score

5.6. Model Performance Results

Table 5: Model Performance Comparison

Model	Accuracy	F1 Score	Precision	Recall
GradientBoosting	**0.945**	**0.949**	**0.925**	**0.975**
RandomForest	0.924	0.932	0.882	0.988
ExtraTrees	0.913	0.923	0.866	0.989

Winner: GradientBoosting with F1 = 0.949.

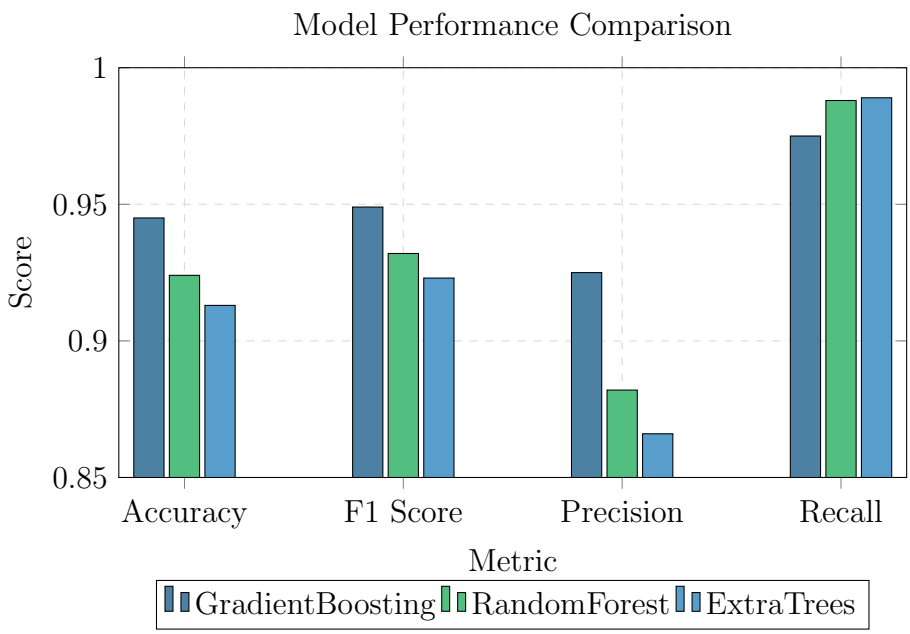

Figure 7: Bar chart: model performance comparison (generated from MLflow results)

6. CI/CD Pipeline

CI/CD is implemented using GitHub Actions with a self-hosted runner on EC2. Triggers automatically on every push to `main`.

6.1. Pipeline Overview

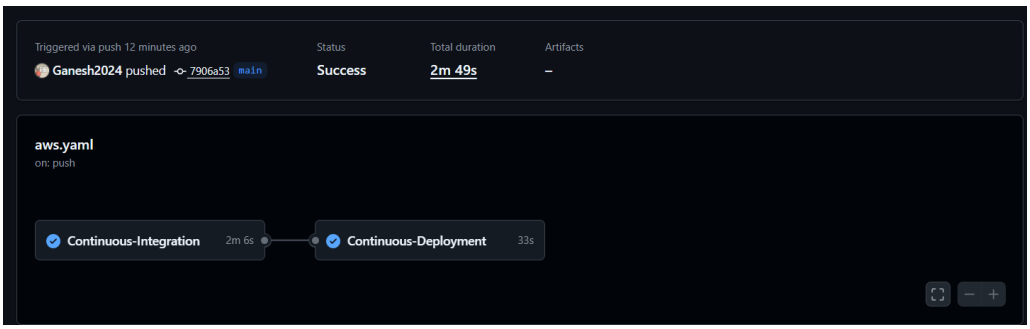

Figure 8: GitHub Actions — CI (2m 6s) + CD (33s) = Total 2m 49s

6.2. Continuous Integration

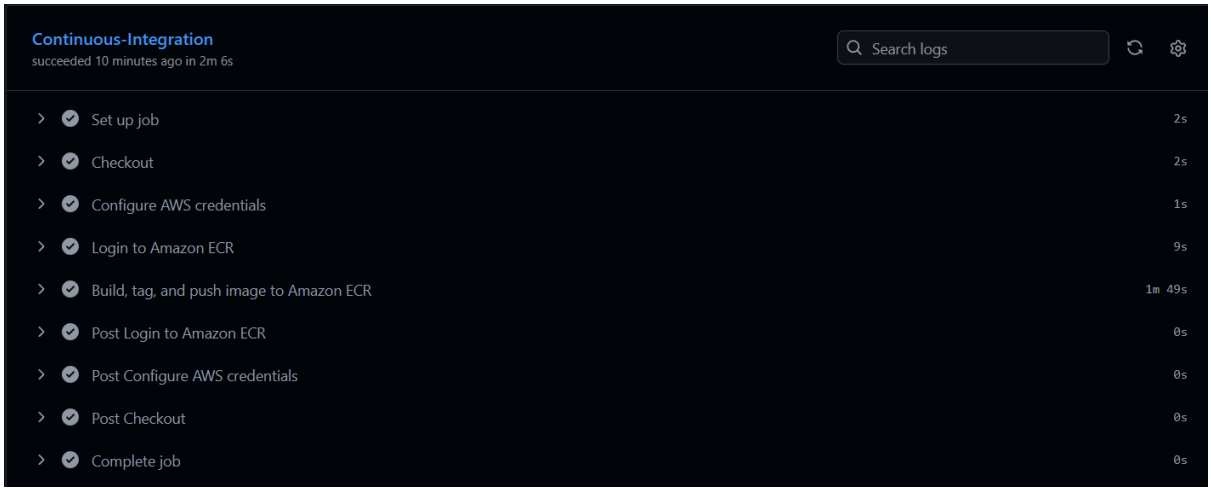

Figure 9: Continuous-Integration — Build, tag, and push Docker image to Amazon ECR

Steps: Checkout → Configure AWS → Login ECR → Build Docker image (1m 49s) → Push to ECR with `latest` tag.

6.3. Continuous Deployment

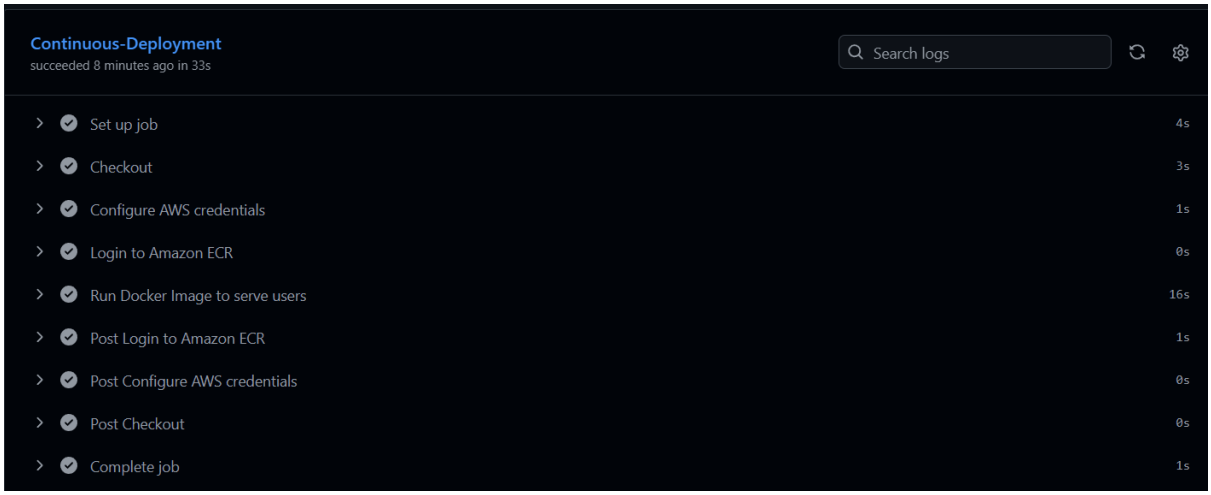

Figure 10: Continuous-Deployment — Pull image from ECR and run container on EC2

Steps: Checkout → Configure AWS → Login ECR → Pull image → Run container on port 5000.

GitHub Secrets: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_DEFAULT_REGION`, `ECR_REPO`, `MONGODB_URL`

7. Web Application

7.1. FastAPI Endpoints

Table 6: API Endpoint Specifications

Method	Endpoint	Response	Description
GET	/	text/html	Renders prediction form
POST	/	text/html	Returns Response-Yes or Response-No
GET	/train	text/plain	Triggers full 6-stage training pipeline

7.2. Prediction Flow

1. User submits 11-field form via browser
2. `DataForm` extracts form fields
3. `VehicleData` builds a 1-row DataFrame
4. `VehicleDataClassifier` loads `MyModel` from S3
5. `MyModel.predict()`: ColumnTransformer → GradientBoosting
6. Returns **Response-Yes** (1) or **Response-No** (0)

8. Software Engineering

8.1. Design Principles

Loose Coupling: Frontend and backend communicate only via REST API. Each pipeline stage is independent.

Single Responsibility: Each component class has exactly one job. Config and Artifact dataclasses cleanly separate concerns.

Fail Fast: Custom `MyException` captures filename and line number of every error. All components wrap logic in `try/except` blocks.

8.2. Logging

- **Handler:** RotatingFileHandler — 5MB max, 3 backup files
- **Format:** `[ timestamp ] name - level - message`
- **Coverage:** Entry/exit logged for every major method

8.3. Exception Handling Pattern

Listing 1: Standard exception handling across all components

```
try:
    # component logic
    return artifact
except Exception as e:
    raise MyException(e, sys) from e
```

9. Known Limitations & Future Work

Table 7: Limitations and Proposed Solutions

Limitation	Proposed Solution
No DVC data versioning	Integrate DVC with S3 remote
No Prometheus/Grafana	Add /metrics endpoint + Grafana dashboard
No docker-compose	Separate frontend/backend as two services
No unit tests	Add pytest suite per component
Airflow local only	Deploy on EC2 or AWS MWAA
Manual training trigger	Schedule DAG with cron expression

10. Conclusion

This project demonstrates a production-grade MLOps system covering the complete machine learning lifecycle. Key achievements:

- **Multi-model comparison:** Three ensemble models compared in MLflow; GradientBoosting achieved F1 = 0.949
- **Automated evaluation:** New models compared against production with configurable 0.02 threshold
- **Pipeline orchestration:** Airflow DAG with proper failure propagation
- **Full CI/CD:** GitHub Actions pipeline deploys in under 3 minutes
- **Experiment tracking:** MLflow logs 5 runs per training cycle
- **Production serving:** FastAPI on AWS EC2 via Docker

AI Usage Disclosure

Artificial Intelligence Tools — Usage Disclosure

In accordance with academic integrity guidelines, I disclose that AI-assisted tools were used in a limited and supportive capacity. The use was strictly restricted to **manual, non-code-generation purposes only**.

Permitted uses of AI tools in this project:

- **Grammar and language correction:** Prompts such as *"Check the grammar of this paragraph"* and *"Rephrase this sentence more clearly"* were used to improve readability of written documentation.
- **Clarification of concepts:** Prompts such as *"Explain what SMOTEENN does in simple terms"* and *"What is the difference between StandardScaler and MinMaxScaler?"* were used to aid understanding of theoretical concepts.
- **Debugging assistance:** Prompts such as *"What does this error mean: columns are missing id?"* were used to understand error messages, not to generate fixes automatically.
- **Report formatting:** Prompts such as *"How do I write a LaTeX table with booktabs?"* were used for document formatting guidance only.

AI tools were NOT used for:

- Writing, generating, or completing any Python code files
- Generating the pipeline architecture or design decisions
- Automating model selection or hyperparameter tuning
- Writing test cases or producing test results

All code files, system design decisions, architecture choices, pipeline structure, model selection rationale, and experimental results are the original work of the author.

Ganesh Mula
Student ID: da25m019
April 2026